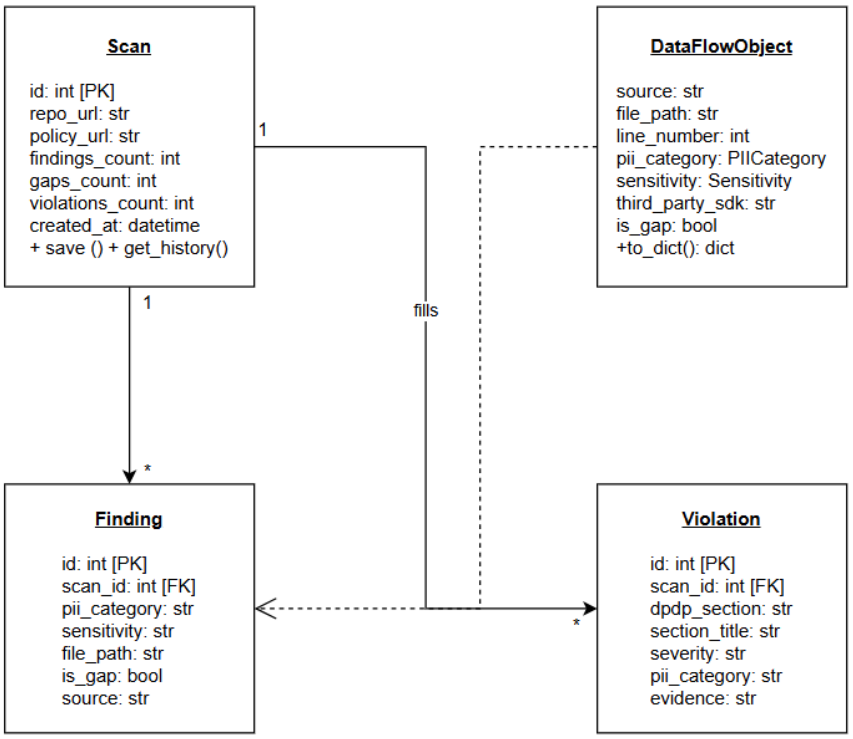
1. Class Diagram:



The 4 classes

Scan — <<entity>>

This is the database table that records every scan that runs. Every time someone clicks "Run Compliance Scan" in the UI, one Scan row is created. It stores:

- id — auto-generated primary key, uniquely identifies this scan
- repo_url — the GitHub repo URL the user entered
- policy_url — the privacy policy URL the user entered
- findings_count — total PII findings found in the code (e.g. 992)
- gaps_count — how many consent gaps were detected (e.g. 3)
- violations_count — how many DPDP violations were flagged (e.g. 5)
- created_at — timestamp of when the scan ran

The <<entity>> stereotype means this is a persistent database entity it maps directly to the scans table in PostgreSQL.

DataFlowObject — <<pydantic model>>

This is NOT a database table. It's a Pydantic schema, an in-memory data structure that lives only during a scan. Every line of code that contains PII, and every privacy policy clause that mentions PII, gets wrapped into a DataFlowObject.

It stores:

- source where it was found. Either "code_scanner" or "presidio_scanner" or "policy_parser"
- file_path which file the finding came from (e.g. care/emr/admin/patient.py)
- line_number exact line number in the file
- pii_category what type of PII was detected (from the PIICategory enum: aadhaar, pan, phone, email, health, biometric etc.)
- sensitivity either "sensitive" (aadhaar, health, biometric, financial) or "personal" (name, phone, email)
- third_party_sdk if the PII is being sent to Firebase, Twilio, Mixpanel etc., that SDK name is stored here. Otherwise null.
- is_gap set to True later by the Consent Gap Detector if this PII type was found in code but NOT in the privacy policy

The <<pydantic model>> stereotype means this class is validated by Pydantic if you try to create one with wrong types, it throws an error immediately.

Finding — <<entity>>

This is a database table, the findings table. It's the persisted version of DataFlowObject. After the scanner produces DataFlowObject instances in memory, the first 200 of them get saved to PostgreSQL as Finding rows.

It stores:

- id — primary key
- scan_id [FK] — foreign key linking back to the Scan that produced this finding
- source — code_scanner or presidio_scanner
- pii_category — aadhaar, health, phone etc.
- sensitivity — sensitive or personal
- file_path — which file
- is_gap — True/False (was this PII type missing from the policy?)

The key difference from DataFlowObject: Finding is permanently stored in the database. DataFlowObject is temporary it exists only during the scan execution.

Violation — <<entity>>

Also a database table, the violations table. Every DPDP rule violation detected by the rule engine gets saved here. Each violation is linked to the scan that found it.

It stores:

- id — primary key
- scan_id [FK] — foreign key linking back to Scan
- dpdp_section — which DPDP section was violated (e.g. "Section 6", "Section 8(3)", "Section 8(4)")
- section_title — human-readable name (e.g. "Consent Requirements", "Purpose Limitation", "Data Minimisation")
- severity — "critical" or "high" or "medium"
- pii_category — which PII type triggered this violation

- evidence — the full explanation text shown in the UI (e.g. "'health' data is collected at care/emr/admin/patient.py line 32 but NOT mentioned in the privacy policy")

The 3 relationships

Relationship 1 — Scan 1 → * Finding (solid vertical line)

This is a **one-to-many composition** relationship. One scan produces many findings. The 1 near Scan means one scan. The * near Finding means zero or more findings. In code this maps to:

```
findings = db.query(Finding).filter(Finding.scan_id == scan_id).all()
```

The foreign key scan_id on Finding enforces this in the database. If you delete a Scan, all its Findings should also be deleted (cascade delete).

Relationship 2 — Scan 1 → * Violation (solid L-shaped line through the middle gap)

Same pattern — one-to-many composition. One scan produces many violations. The line routes through the gap between the two columns because a straight diagonal would cut through one of the boxes, which is not allowed in UML diagrams. The 1 is near Scan's right edge, the * is near Violation's left edge. In code:

```
violations = db.query(Violation).filter(Violation.scan_id == scan_id).all()
```

Again enforced by the scan_id foreign key on Violation.

Relationship 3 — DataFlowObject - - uses - → Finding (dashed diagonal line)

This is a **dependency** relationship, shown with a dashed line. It means Finding is derived from or filled by DataFlowObject. They share the same fields (source, file_path, pii_category, sensitivity etc.) because Finding is essentially the database-persisted form of DataFlowObject.

In code this happens here:

```
for f in code_findings:          # f is a DataFlowObject
    db.add(Finding(
        scan_id      = scan.id,
        source       = f.source,          # copied from DataFlowObject
        file_path    = f.file_path,       # copied from DataFlowObject
        pii_category = f.pii_category.value,
        sensitivity  = f.sensitivity.value,
        is_gap       = f.pii_category in gap_categories,
    ))
```

The dashed line means "depends on" — Finding wouldn't exist without DataFlowObject. But it's not a composition (not a solid line) because DataFlowObject doesn't own Finding — they just share data.

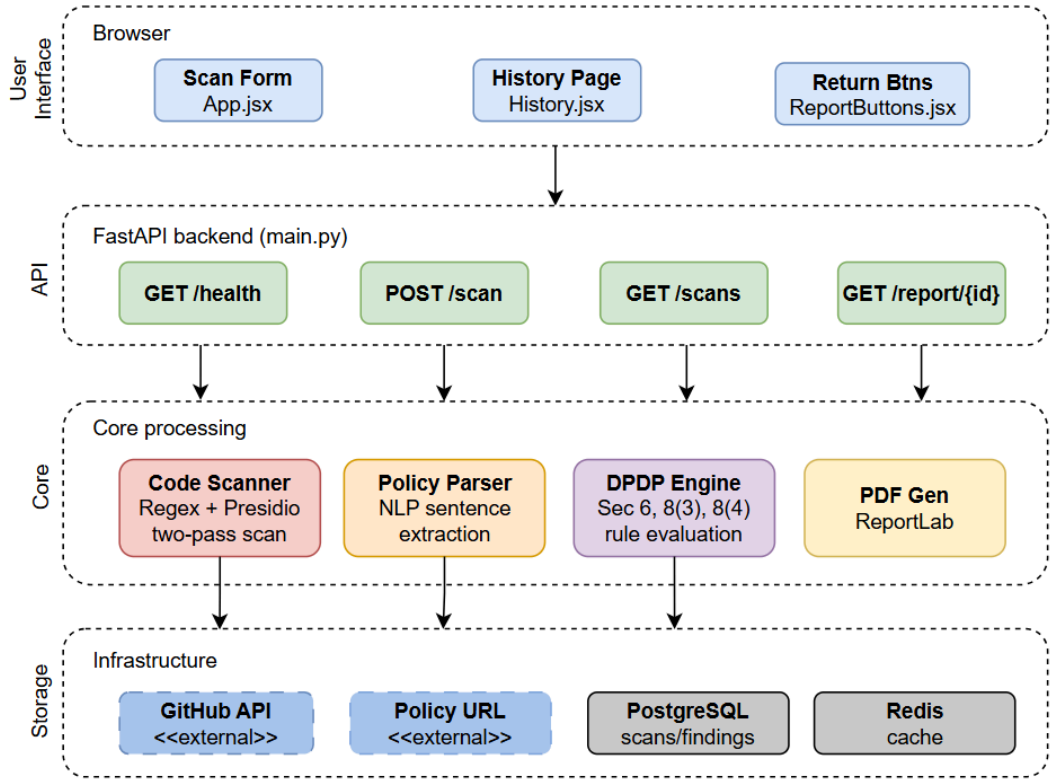
Why Scan owns both Finding and Violation but they don't directly relate to each other

Finding and Violation are **siblings** under Scan — they are both children of the same parent but have no direct relationship to each other. This is by design:

- Finding is raw evidence — "this PII exists in this file at this line"
- Violation is the legal judgement — "this PII being here violates Section 6 of the DPDP Act"

The DPDPRuleEngine reads all Findings, processes them against the rules, and produces Violations. That processing step is the bridge between the two — but it happens in memory (not in the database), so no direct database relationship exists between Finding and Violation.

2. Component Diagram:



The 4 layers

The diagram is divided into 4 horizontal layers, each representing a different level of the system. Each layer is shown as a dashed rectangle because dashed borders in UML mean "boundary" or "subsystem."

Layer 1 — Browser (UI layer)

This is everything that runs inside the user's web browser. It has 3 components:

Scan Form (App.jsx)
The main page the user sees. It contains the 3 input fields (GitHub repo URL, privacy policy URL, GitHub token) and the Run Compliance Scan button. When the user clicks the button, this component sends a POST request to the FastAPI backend and displays the results — compliance score, consent gaps, violations list, and the report download buttons.

History Page (History.jsx)
The /history route in the browser. It fetches all past scans from the GET /scans endpoint and displays them as cards showing repo name, finding count, gap count, and violation count. Each card has a View Report button and a Download PDF button.

Report Buttons (ReportButtons.jsx)
A small reusable component used inside both App.jsx and History.jsx. It renders the "View Full Report" (dark button, opens HTML report in new tab) and "Download PDF" (red button, triggers PDF download) buttons for a given scan ID.

Layer 2 — FastAPI Backend (API layer)

This is the Python server running inside Docker on localhost:8000. It has 4 endpoint components:

GET /health
Simple health check. Returns {"status": "ok", "message": "PrivacyPilot is running"}. Used by the frontend to verify the backend is reachable. Called at startup.

POST /scan
The most important endpoint. Receives the repo URL, policy URL, and GitHub token. It orchestrates the entire scan pipeline — calls the code scanner, calls the policy parser, runs gap detection, runs the DPDP rule engine, calculates the compliance score, saves everything to PostgreSQL, and returns the full results as JSON. This is the endpoint that takes 2–3 minutes to respond.

GET /scans
Returns the full history of all past scans as a JSON array. Each entry contains scan ID, repo URL, timestamp, findings count, violations count, and gaps count. Called by the History page on load.

GET /report/{id}
Serves the compliance report for a specific scan. Without query parameter → returns an HTML page with the full compliance report. With ?format=pdf → generates and streams a PDF file using ReportLab. The report includes score, summary, consent gaps, violations with evidence and fixes, and a sample findings table.

Layer 3 — Core Processing (business logic layer)

This is where all the intelligence lives. 4 components:

Code Scanner
The GitHub repository scanner. It uses PyGitHub to clone/access the repo, then scans all .py, .js, and .ts files in two passes:

- Pass 1 — Regex patterns match PII variable names (phone_number, aadhaar_number, blood_group, fingerprint_hash etc.) and third-party SDK calls (firebase, twilio, mixpanel etc.)
- Pass 2 — Microsoft Presidio NER runs on every unmatched line to catch PII that regex missed (actual Aadhaar number values, phone number formats, person names etc.)

Each matched line becomes a DataFlowObject. Currently capped at 300 files for speed. Returns a list of DataFlowObject instances.

Policy Parser
The privacy policy text extractor. Downloads the privacy policy from the given URL (HTML page or raw text), strips HTML tags, splits into sentences, and looks for sentences that mention PII collection using pattern matching. Produces a list of DataFlowObject instances where source="policy_parser" — one per PII category mentioned in the policy.

DPDP Engine
The legal rule evaluator. Takes the code findings, policy claims, and the set of gap categories (PII types in code but not in policy) and evaluates 3 rules:

- Section 6 (Consent Requirements) — is sensitive PII collected without being disclosed?
- Section 8(3) (Purpose Limitation) — is PII being sent to analytics SDKs like Firebase or Mixpanel?
- Section 8(4) (Data Minimisation) — is sensitive PII collected beyond what's in the policy?

For each rule that fires, it creates a DPDPViolation object with section, severity, PII category, full evidence text, and a remediation recommendation.

PDF Generator
Takes the scan results (violations, findings, compliance score) and generates a formatted PDF using the ReportLab library. Produces a multi-page document with a score gauge, scan summary table, consent gaps list, violations section (with evidence and fixes), and a sample code findings table. Returns the PDF as bytes which FastAPI streams to the browser.

Note: WeasyPrint was the original choice but was dropped because it requires the gobject system library which is not available in the python:3.11-slim Docker image. ReportLab has no system dependencies and works everywhere.

Layer 4 — Infrastructure (storage and external)

GitHub API (external, dashed border)
Not a component we built — it's GitHub's API accessed via the PyGitHub library. The Code Scanner authenticates using the user's personal access token and calls GitHub's REST API to read the repository file tree and individual file contents. Shown with a dashed border to indicate it's external to our system.

Privacy Policy URL (external, dashed border)
The URL of the company's privacy policy page. The Policy Parser makes an HTTP GET request to this URL to fetch the policy text. Also external — we don't control it.

PostgreSQL
Our primary database. Stores 3 tables: scans (one per scan run), findings (up to 200

PII findings per scan), and violations (all DPDP violations per scan). Runs in the db Docker container on port 5432. Persists all data across restarts.

Redis
In-memory cache. Currently used for caching but has capacity for future Celery async job queueing (listed as future work). Runs in the redis Docker container on port 6379.

The 8 arrows and what they mean

Arrow 1 — Browser → FastAPI (HTTP REST)
The React frontend calls the FastAPI backend using standard browser fetch() calls. The single most important call is POST /scan. Others include GET /scans (history page) and GET /report/{id} (report). All calls go over HTTP on localhost since both run on the same machine. Cross-origin requests are handled by the CORS middleware configured in main.py.

Arrows 2, 3, 4, 5 — FastAPI → 4 Core Components
These 4 arrows fan out from FastAPI to each processing component. They show function calls, not HTTP calls — these are all Python imports within the same Docker container:

- FastAPI imports and calls scan_github_repo() from code_scanner.py
- FastAPI imports and calls parse_privacy_policy() from policy_parser.py
- FastAPI imports and calls run_dpdp_rules() from rule_engine.py
- FastAPI imports and calls generate_pdf() from pdf_generator.py

Arrow 6 — Code Scanner → GitHub API
An outbound HTTPS call from inside the Docker container to api.github.com. Authenticated using the Bearer token the user provides. This is the call that takes the most time — one API call per file, 300 files = 300 API calls.

Arrow 7 — Policy Parser → Privacy Policy URL
An outbound HTTPS call to whatever URL the user entered (e.g. https://parth268.github.io/...). A simple requests.get() call. Fast — single HTTP request.

Arrow 8 — DPDP Engine → PostgreSQL
The save operation. After the engine generates violations, FastAPI saves them (along with the scan summary and findings) to PostgreSQL using SQLAlchemy ORM. This goes through Docker's internal network from the backend container to the db container.

How the 4 layers work together in a single scan

When the user clicks "Run Compliance Scan":

1. **Browser** sends POST /scan to **FastAPI**
2. **FastAPI** calls **Code Scanner** → which calls **GitHub API** → returns 992 DataFlowObjects
3. **FastAPI** calls **Policy Parser** → which calls **Privacy Policy URL** → returns 6 DataFlowObjects
4. **FastAPI** computes gap categories in memory (no component needed)
5. **FastAPI** calls **DPDP Engine** → which evaluates rules → returns 5 DPDPViolations
6. **FastAPI** saves everything to **PostgreSQL**
7. **FastAPI** returns JSON to **Browser**
8. When user clicks Download PDF → **FastAPI** reads from **PostgreSQL** → calls **PDF Generator** → streams PDF to **Browser**

Every scan follows this exact sequence through the 4 layers.

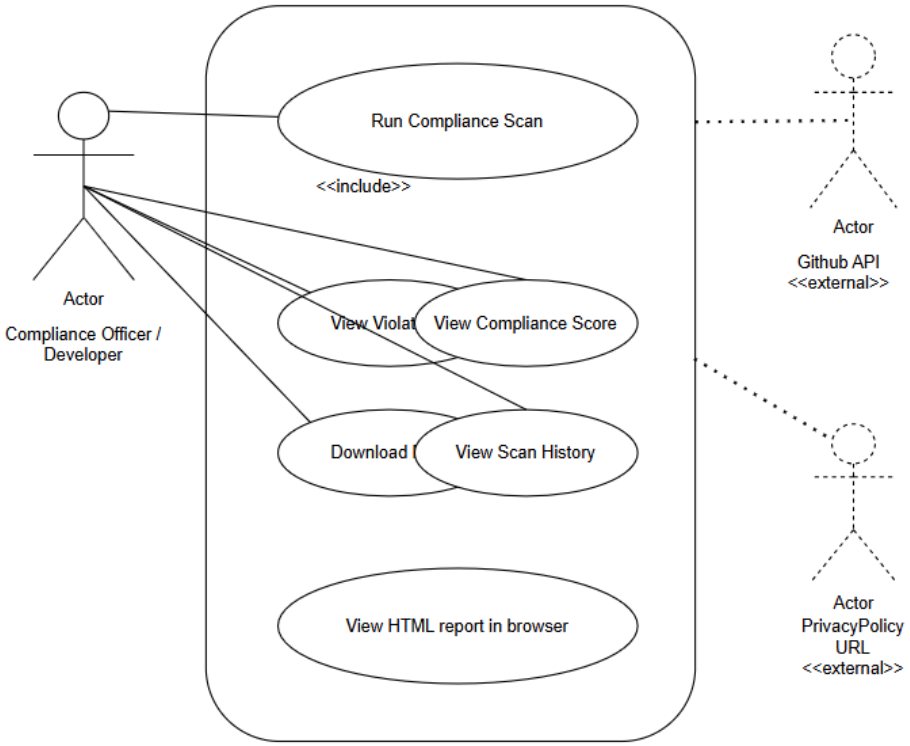
Why this architecture matters for DPDP compliance

The separation into these 4 layers means:

- The **Code Scanner** can be upgraded independently (add more regex patterns, better Presidio config) without touching the API or UI
- The **DPDP Engine** can add new provisions (Section 7, Section 9 etc.) without changing how scanning works
- The **PDF Generator** can be completely replaced (e.g. with a different library) without affecting anything else
- The **database schema** can evolve independently of the UI

This is the modularity argument you can make to your professor when explaining your architecture choices.

3. Use Case:



The system boundary encompasses all functionality implemented within the PrivacyPilot platform. External actors GitHub API and Privacy Policy URL participate in the scan use case as data providers. The platform does not store GitHub credentials beyond the duration of a single scan request, and no data is persisted from external systems except the scan findings and violations stored in the local PostgreSQL database. All outputs of the system are for research demonstration purposes and do

A use case diagram shows the system from the **user's perspective** — who uses the system, what can they do with it, and what external systems does it interact with. It does NOT show how things work internally (that's the sequence and activity diagrams). It only shows **what** the system does, not **how**.

The three elements are:

- **Actors** — stick figures representing who interacts with the system
- **Use cases** — ovals representing what the system can do
- **System boundary** — the large rectangle labeled "PrivacyPilot System" that separates internal functionality from external actors

The 3 Actors

Actor 1 — Compliance Officer / Developer (solid stick figure, left side)

This is the primary human actor. In your project this represents:

- A startup CTO who wants to check if their app is DPDP compliant
- A developer who wants to scan their own codebase before deployment
- A security auditor doing a compliance review

This actor is shown as a solid stick figure (not dashed) because they are a direct, intentional user of the system. They initiate every scan and consume every report. They interact with the React UI on localhost:5174.

All 6 use cases are accessible to this actor — they can run scans, view violations, view the compliance score, download PDF reports, view scan history, and view HTML reports in the browser.

Actor 2 — GitHub API (dashed stick figure, top-right)

This is an external system actor — shown with a dashed border to indicate it is outside our system and we do not control it. GitHub is an actor because it actively participates in the scan process — it responds to our API calls with file contents. Without GitHub responding, the scan cannot complete.

It interacts only with the "Run compliance scan" use case — specifically the code scanning step inside it. It receives authenticated requests (Bearer token) and returns file tree data and file contents via REST API responses.

The dashed border follows UML convention for external/secondary actors that the system depends on but does not control.

Actor 3 — Privacy Policy URL (dashed stick figure, bottom-right)

Also an external system actor. This represents any web server that hosts a company's privacy policy. For example:

- *https://parth268.github.io/App-Privacy-Policy/PrivacyPolicy.html*
- *https://raw.githubusercontent.com/privacypilot-capstone/healthtech-demo/main/PRIVACY_POLICY.md*

It participates in the scan by responding to an HTTP GET request with policy text. Like GitHub API, it interacts only with "Run compliance scan" — specifically the policy parsing step.

The reason it is modeled as an actor (not just a component) is that it is an independent entity outside our system boundary — we depend on it but cannot control whether it responds, what format it returns, or whether the page requires JavaScript rendering.

The 6 Use Cases

Use case 1 — Run compliance scan (center-top oval)

This is the **primary use case** and the most complex one. It is triggered by the Compliance Officer clicking "Run Compliance Scan" in the UI after entering the repo URL, policy URL, and GitHub token.

What happens inside this use case (abstracted):

- GitHub repository is scanned for PII
- Privacy policy is downloaded and parsed
- Consent gaps are detected
- DPDP rules are evaluated
- Compliance score is calculated
- Results are saved to the database

This use case depends on both external actors — GitHub API must respond with file data, and the Privacy Policy URL must respond with policy text. Both external actors are connected to this use case via dashed association lines.

This use case is connected to all other 4 remaining use cases via <<include>> relationships (explained below) — meaning when you run a scan, you automatically get violations, score, report, and history record.

Use case 2 — View violations (left-center oval)

After a scan completes, the compliance officer can see all DPDP violations displayed in the React UI. Each violation shows:

- Which DPDP section was violated (Section 6, 8(3), or 8(4))
- Severity (CRITICAL or HIGH)
- Which PII type triggered it
- The evidence text (file name and line number)
- The recommended fix

This is also accessible separately via the API endpoint GET /scans/{id}/violations for programmatic access.

Use case 3 — View compliance score (right-center oval)

The compliance score (0–100) is displayed as the large colored number at the top of the scan results. The scoring logic deducts 15 points per critical violation, 8 per high, 4 per medium, and 5 per consent gap, with a floor of 0. The rating (Good / Moderate / Poor / Critical) and color (green / orange / red / dark red) update automatically based on the score.

This is also returned in every scan response JSON under the compliance key so the frontend can render it without a separate API call.

Use case 4 — Download PDF report (left-lower oval)

The compliance officer can download a professionally formatted PDF report for any scan. The PDF is generated on demand by the GET /report/{id}?format=pdf endpoint using ReportLab. It contains:

- Compliance score with color-coded rating
- Scan summary (repo, date, finding count, violations count)
- Consent gaps section
- Full violations section with evidence and fixes
- Sample code findings table
- Disclaimer (not legal advice)

This is the deliverable the compliance officer would hand to their legal team or management.

Use case 5 — View scan history (right-lower oval)

The History page at /history shows all past scans stored in the database. Each entry shows the repo name, timestamp, number of findings, number of gaps, and number of violations (in red if > 0). Each entry also has "View Report" and "Download PDF" buttons.

This is powered by the GET /scans endpoint which queries the scans table in PostgreSQL ordered by created_at descending (newest first).

Use case 6 — View HTML report in browser (bottom oval)

Different from downloading a PDF — this opens the HTML compliance report directly in a new browser tab at localhost:8000/report/{id}. It is the same content as the PDF but rendered as a styled webpage. Useful for quickly reading the report without downloading a file. The HTML report is generated by report_generator.py and served by FastAPI as an HTMLResponse.

The relationships between use cases

<<include>> relationships (dashed lines between use cases)

In UML, <<include>> means one use case always includes another — it is a mandatory part of it. You cannot do the base use case without the included one happening.

In the diagram, "Run compliance scan" includes all other use cases via a chain of <<include>> lines:

Run compliance scan

↓ <<include>>

View violations + View compliance score

↓ <<include>>

Download PDF report + View scan history

↓ <<include>>

View HTML report in browser

What this means in practice:

- Every time you run a scan, violations ARE generated (you cannot run a scan without violations being calculated — even if the result is 0 violations)
- Every time you run a scan, a compliance score IS calculated
- Every scan IS saved to history (database always records it)
- Every scan HAS a report available (HTML and PDF are always generatable from stored data)

The <<include>> is appropriate here because these are not optional features — they are mandatory outputs of every scan.

The association lines (solid lines from actor to use cases)

Compliance Officer → Run compliance scan
The officer initiates the scan by filling the form and clicking the button. This is the most important association.

Compliance Officer → View violations
The officer reads the violations list displayed in the React UI after the scan.

Compliance Officer → View compliance score
The officer sees the score gauge (31/100 — Critical) at the top of the results.

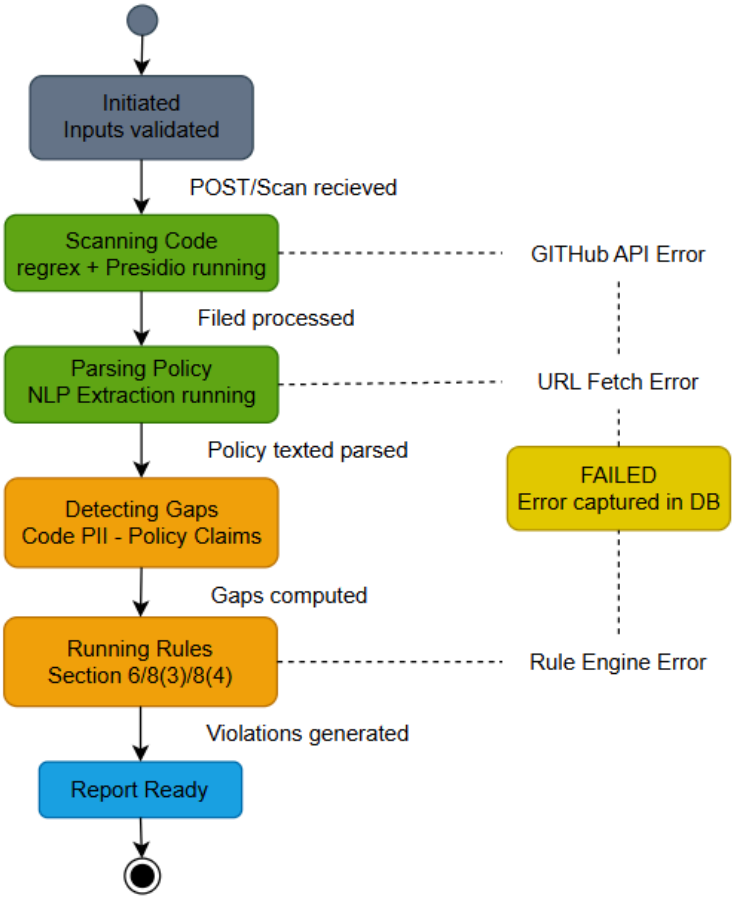
Compliance Officer → Download PDF report
The officer clicks the red "Download PDF" button — either in the scan results or in the History page.

Compliance Officer → View scan history
The officer navigates to /history to see all past scans.

GitHub API → Run compliance scan (dashed line)
GitHub participates as a data provider — it sends file contents when the scanner calls it. The dashed line shows this is an external, system-to-system interaction rather than a direct user action.

Privacy Policy URL → Run compliance scan (dashed line)
The policy URL server participates by responding to the HTTP GET with policy text. Same reasoning as GitHub.

4. State Machine Diagram



A state machine diagram shows the **lifecycle of one object** — how it changes from one state to another in response to events. For PrivacyPilot, the object being tracked is a **Scan**. From the moment a user clicks "Run Compliance Scan" to the moment results appear on screen, the Scan passes through multiple distinct states. This diagram captures every state, every transition, and every possible failure point.

Think of it like a traffic light — it can only be in one state at a time (Red, Yellow, or Green), and it moves between states when something happens. A Scan is the same — it can only be in one state at a time, and it moves forward when each processing step completes, or sideways into "Failed" when something goes wrong.

The symbols used

Filled black circle (top) — Initial pseudostate. This is not a real state — it is the starting point, the moment before the Scan object exists. The arrow coming out of it points to the first real state.

Rounded rectangles — Real states. Each one represents a period of time during which the Scan object is doing something specific (or waiting for something to complete).

Arrows with labels — Transitions. Each arrow shows what event or condition causes the Scan to move from one state to the next. The label on the arrow is the trigger.

Dashed arrows going right — Error transitions. These show what happens when something goes wrong during a particular state. All error paths lead to the same "Failed" state.

Circle with inner filled circle (bottom) — Final state. This is where the Scan lifecycle ends — successfully.

The 7 states in order

State 1 — Initiated

What it means: The Scan object has been created. The user has submitted the form and the POST /scan request has been received by FastAPI.

What happens in this state:

- FastAPI validates the request body using Pydantic (checks that github_repo_url, privacy_policy_url, and github_token are all present and non-empty)
- The three input values are stored in the ScanRequest object in memory
- A database Scan row is NOT created yet — the system waits until all scanning is complete before writing to the database

Transition out: POST /scan received — as soon as the inputs are validated, the scan immediately moves to the next state. There is no waiting in this state.

What it looks like in code:

```
@app.post("/scan")
def run_scan(req: ScanRequest, db: Session = Depends(get_db)):
    # Scan is now in "Initiated" state
    # req.github_repo_url, req.privacy_policy_url, req.github_token
    # have all been validated by Pydantic
```

Can it fail here? Yes — if the request body is malformed or missing fields, FastAPI returns a 422 Validation Error before even reaching this state. But that is a pre-state failure, not a state transition.

State 2 — Scanning code

What it means: The code scanner is actively reading files from the GitHub repository and searching for PII.

What happens in this state:

- scan_github_repo() is called with the repo URL and GitHub token
- PyGitHub authenticates with GitHub using the token
- The full file tree of the repository is retrieved
- Up to 300 .py, .js, and .ts files are identified
- For each file, the scanner:
    - Downloads the file content via GitHub API
    - Runs Pass 1 (regex patterns) on every line
    - Runs Pass 2 (Microsoft Presidio NER) on unmatched lines
- Each matching line becomes a DataFlowObject in memory
- Third-party SDK calls (firebase, twilio, mixpanel etc.) are identified and recorded

Duration: This is the **longest state** — typically 2–4 minutes because it makes one GitHub API call per file (up to 300 sequential calls).

Transition out: files processed — when all 300 files have been scanned and the list of DataFlowObject instances is complete, the scan moves to the next state.

Error transition → Failed: GitHub API error — if the GitHub token is invalid, the token has expired, the repo is private and not accessible, GitHub's API is down, or the rate limit is exceeded (60 requests/hour for unauthenticated, 5000/hour for authenticated tokens), the scanner throws an exception. The scan moves to "Failed."

What it looks like in code:

```
# Scan is now in "Scanning code" state
code_findings = scan_github_repo(
    req.github_repo_url, req.github_token
)
# State exits when this line completes
# If exception thrown → transition to Failed
```

State 3 — Parsing policy

What it means: The policy parser is downloading and extracting PII claims from the privacy policy document.

What happens in this state:

- parse_privacy_policy() is called with the policy URL
- An HTTP GET request is made to the URL using the requests library
- HTML tags are stripped from the response
- <script> and <style> blocks are removed first (to avoid false matches in JavaScript code)
- The remaining text is split into sentences
- Each sentence is checked for:
    - Collection context words (collect, process, store, share, disclose, transfer)

    - PII keywords (name, email, phone, location, aadhaar, health, biometric, pan, financial, birth)
- Matching sentences become DataFlowObject instances with source="policy_parser"
- Duplicate PII categories are deduplicated (one entry per category)

Duration: Fast — typically 1–3 seconds. Single HTTP request plus text processing.

Transition out: policy text parsed — when all sentences have been processed and the list of policy claims is complete.

Error transition → Failed: URL fetch error — if the URL is unreachable (404, timeout, connection refused), the website requires JavaScript rendering (returns empty HTML shell), or the response is in an unreadable format, an exception is raised. The scan moves to "Failed."

What it looks like in code:

```
# Scan is now in "Parsing policy" state
policy_claims = parse_privacy_policy(req.privacy_policy_url)
# State exits when this line completes
# If exception thrown → transition to Failed
```

State 4 — Detecting gaps

What it means: The Consent Gap Detector is comparing code-derived PII types against policy-claimed PII types.

What happens in this state:

- All unique PII categories found in the code are collected into a set: code_pii
- All unique PII categories mentioned in the policy are collected into a set: policy_pii
- Set subtraction is performed: gap_categories = code_pii - policy_pii
- The result is the set of PII types that appear in the code but are **NOT** mentioned in the privacy policy — these are the consent gaps
- Each DataFlowObject from the code scan has its is_gap field updated — True if its pii_category is in gap_categories, False otherwise

Example from Scan 2 (healthtech-demo):

code_pii = {aadhaar, pan, phone, email, location, health, biometric, date_of_birth}

policy_pii = {name, email, phone, address, financial}

gaps = {aadhaar, pan, health, biometric, date_of_birth}

Duration: Extremely fast — microseconds. Pure in-memory Python set operations.

Transition out: gaps computed — immediately after the set subtraction completes.

Can it fail? Almost never — it is pure Python with no external calls. Only fails if memory is exhausted (not realistic for normal scan sizes).

What it looks like in code:

```
# Scan is now in "Detecting gaps" state
code_pii       = {f.pii_category for f in code_findings}
policy_pii     = {f.pii_category for f in policy_claims}
gap_categories = code_pii - policy_pii
# State exits immediately — pure computation
```

State 5 — Running rules

What it means: The DPDP Rule Engine is evaluating all three legal provisions against the scan findings.

What happens in this state:

- run_dpdp_rules() is called with code findings, policy claims, and gap categories
- The engine iterates through every DataFlowObject from the code scan
- For each finding, it checks three rules: **Rule 1 — Section 6 (Consent Requirements):**
  Is this PII category in the gap set (not disclosed in policy)?
  - If yes AND the PII is sensitive (aadhaar, health, biometric, financial, pan) → CRITICAL violation
  - If yes AND the PII is non-sensitive (phone, email, location, name, dob) → HIGH violation
- **Rule 2 — Section 8(3) (Purpose Limitation):**
  Is this PII being sent to an analytics SDK (firebase, mixpanel, amplitude, segment, crashlytics)?
  - If yes → HIGH violation (purpose extends beyond stated service)
  - If yes AND the PII is sensitive → CRITICAL violation (stricter requirements for sensitive data)
- **Rule 3 — Section 8(4) (Data Minimisation):**
  Is sensitive PII being collected but not mentioned in the policy?
  - If yes → HIGH violation (collecting more than necessary for stated purpose)
- Duplicate violations are suppressed (a seen set tracks which section+category combinations have already been flagged)
- Each triggered rule creates a DPDPViolation object with full evidence text and recommendation
- The compliance score is calculated: start at 100, deduct 15 per critical, 8 per high, 4 per medium, 5 per gap, floor at 0

Duration: Fast — milliseconds. In-memory iteration.

Transition out: violations generated — when all findings have been checked against all rules.

Error transition → Failed: rule engine error — if an unexpected exception occurs in the rule logic (rare — only happens if data is malformed).

What it looks like in code:

```
# Scan is now in "Running rules" state
violations = run_dpdp_rules(
    code_findings, policy_claims, gap_categories
)
compliance = calculate_compliance_score(
    violations, len(code_findings), len(gap_categories)
)
# State exits when both lines complete
```

State 6 — Report ready

What it means: All processing is complete. The results are saved to the database and the response is returned to the frontend.

What happens in this state:

- A Scan row is created in PostgreSQL with summary counts
- Up to 200 Finding rows are inserted
- All Violation rows are inserted
- db.commit() is called — this is the point of no return (data is permanently saved)
- The API response JSON is assembled and returned to the React frontend
- The React UI renders the compliance score gauge, gaps list, and violations list
- The "View Full Report" and "Download PDF" buttons become active (they call /report/{scan_id})
- If the user clicks Download PDF → GET /report/{scan_id}?format=pdf is called → ReportLab generates the PDF → streamed back to browser

Transition out: scan_id returned — the API returns 200 OK with the full JSON response. The Scan object reaches its final state.

Can it fail here? Technically yes — if PostgreSQL is down or disk is full, db.commit() throws and db.rollback() is called. But in normal operation this state is the success end point.

What it looks like in code:

```
# Scan is now in "Report ready" state
db.commit()
print(f"[db] Scan {scan.id} saved — {len(violations)} violations")
return {
    "scan_id": scan.id,
    "violations": [...],
    "compliance": {...},
```

```
    ...
}
# HTTP 200 OK returned → Final state reached
```

State 7 — Failed (error state, right side)

What it means: Something went wrong during the scan. The error is captured and returned to the user.

What happens in this state:

- The except Exception as e: block in run_scan() catches the exception
- db.rollback() is called — no partial data is saved to the database
- The error message is returned as HTTP 500 with a detail field
- The React UI displays the error message in red below the scan button

What it looks like in code:

```
except Exception as e:
    db.rollback()          # undo any partial DB writes
    raise HTTPException(
        status_code=500,
        detail=str(e)      # error shown in UI
    )
# Scan is now in "Failed" state — no DB record created
```

Three error transitions into Failed:

1. From Scanning code — GitHub API errors (invalid token, private repo, rate limit)
2. From Parsing policy — URL fetch errors (unreachable URL, JavaScript-rendered page, timeout)
3. From Running rules — rule engine errors (malformed data, unexpected exception)

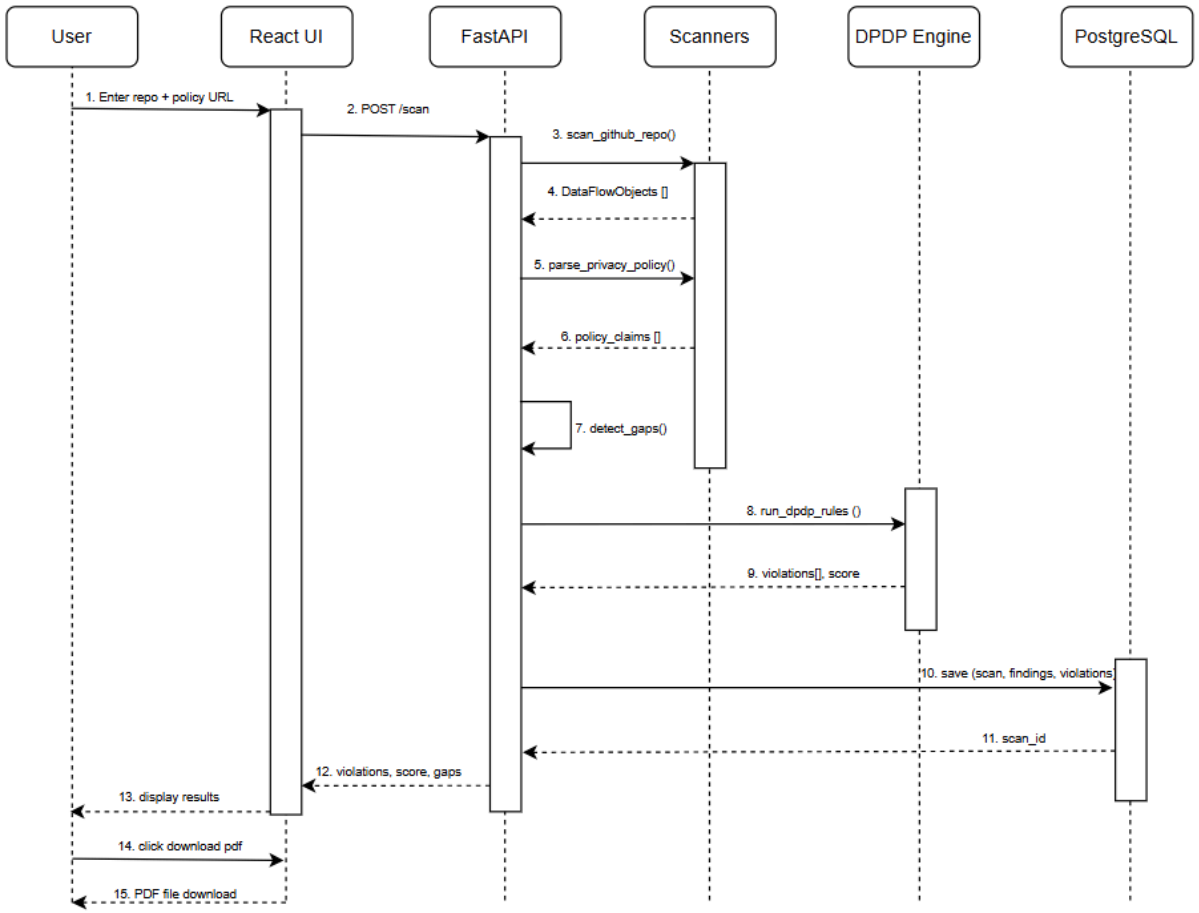
Note: "Detecting gaps" has no error transition shown because it is pure Python computation with no external dependencies — it cannot realistically fail.

The complete transition table

From state	Event / trigger	To state
Initial	POST /scan received, inputs valid	Initiated

Initiated	inputs validated	Scanning code
Scanning code	all files processed	Parsing policy
Scanning code	GitHub API error	Failed
Parsing policy	policy text extracted	Detecting gaps
Parsing policy	URL fetch error	Failed
Detecting gaps	gap set computed	Running rules
Running rules	violations + score generated	Report ready
Running rules	rule engine error	Failed
Report ready	scan_id returned to frontend	Final state

5.



A sequence diagram shows **who talks to whom, in what order, and what they say**. Time flows top to bottom. Each vertical dashed line is a **lifeline** — one participant staying alive throughout the interaction. Horizontal arrows are **messages** — either calls (solid arrows going right) or return values (dashed arrows going left). The narrow rectangles on lifelines are **activation bars** — showing when that participant is actively doing work.

The 6 participants

Participant	What it represents
User	The person at the browser
React UI	The frontend running on localhost:5174

FastAPI	The backend running on localhost:8000
Scanners	CodeScanner + PolicyParser + Presidio (grouped)
DPDP Engine	The rule evaluator in rule_engine.py
PostgreSQL	The database in the db container

The 15 messages explained

Message 1 — User → React UI: enter repo + policy URL
The user fills in the three form fields and clicks the button. No network call yet — just browser interaction.

Message 2 — React UI → FastAPI: POST /scan
The fetch() call fires. Sends the repo URL, policy URL, and token as JSON. The React button changes to "Scanning... (this takes 2–3 min)" and disables.

Message 3 — FastAPI → Scanners: scan_github_repo()
FastAPI calls the code scanner function. The scanner authenticates with GitHub using the token and starts reading files one by one. This is the longest step — up to 300 files, one API call each.

Message 4 — Scanners → FastAPI: DataFlowObjects[]
After all files are processed, the scanner returns a list of DataFlowObjects — each representing one PII finding in the code (992 findings in the OHC CARE scan).

Message 5 — FastAPI → Scanners: parse_privacy_policy()
FastAPI calls the policy parser with the policy URL. The parser makes one HTTP GET request to the URL, strips HTML, splits into sentences, and extracts PII claims.

Message 6 — Scanners → FastAPI: policy claims[]
Parser returns a smaller list of DataFlowObjects — each representing one PII category mentioned in the policy (6 claims in the OHC CARE scan).

Message 7 — FastAPI self-loop: detect_gaps()
FastAPI does this internally — no external call needed. It computes code_pii - policy_pii using Python set subtraction. The loop arrow on the FastAPI lifeline represents this self-contained computation.

Message 8 — FastAPI → DPDP Engine: run_dpdp_rules()
FastAPI passes the findings, policy claims, and gap categories to the rule engine. The engine evaluates Section 6, 8(3), and 8(4) against each finding.

Message 9 — DPDP Engine → FastAPI: violations[], score
The engine returns the list of DPDPViolation objects plus the calculated compliance score (e.g. 31/100 Critical).

Message 10 — FastAPI → PostgreSQL: save(scan, findings, violations)
FastAPI writes everything to the database in one transaction — one Scan row, up to 200 Finding rows, all Violation rows. This is wrapped in a try/except with db.rollback() on failure.

Message 11 — PostgreSQL → FastAPI: scan_id
Database confirms the save and returns the auto-generated scan ID (e.g. scan_id = 2).

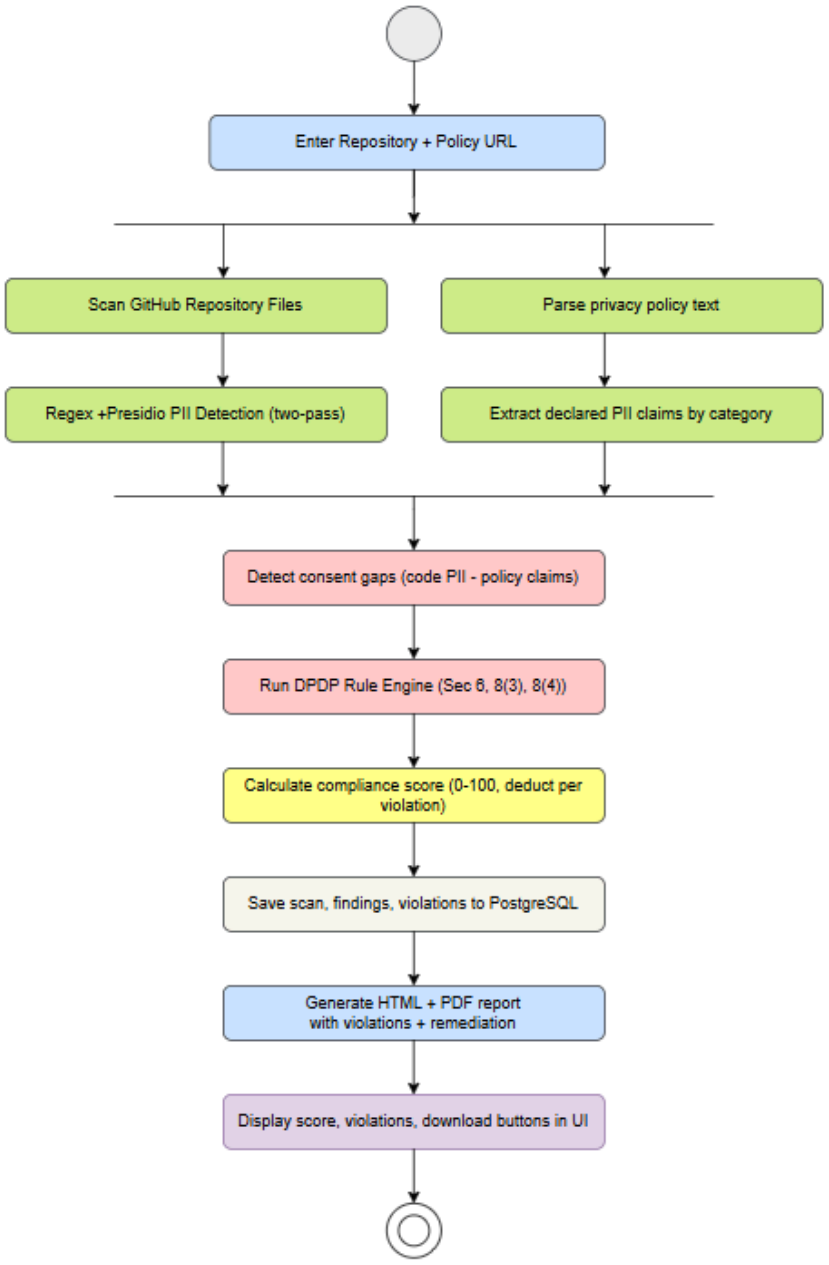
Message 12 — FastAPI → React UI: violations, score, gaps
FastAPI returns the HTTP 200 response with the full JSON payload including violations list, compliance score object, gap categories, and sample findings.

Message 13 — React UI → User: display results
React renders the compliance score gauge, consent gaps, violations cards, and the View Report / Download PDF buttons.

Message 14 — User → React UI: click download PDF
User clicks the red Download PDF button. React opens localhost:8000/report/{scan_id}?format=pdf.

Message 15 — React UI → User: PDF file download
FastAPI reads from the database, generates the PDF using ReportLab, and streams it back. The browser triggers the file download dialog.

6. Activity Diagram:



An activity diagram shows **workflow** — the step by step process the system follows to complete a task. Think of it as a detailed flowchart of what happens inside the system from start to finish. Unlike the sequence diagram (which shows who talks to whom), the activity diagram shows **what the system does at each step and in what order**.

The key difference from a simple flowchart is that activity diagrams can show **parallel activities** — two things happening at the same time — using fork and join bars.

The symbols used

Filled black circle (top) — Start point. The workflow begins here the moment the user clicks Run Compliance Scan.

Rounded rectangles — Activities. Each one represents a specific action the system takes.

Horizontal thick black bars — Fork bar (splits into parallel) and Join bar (waits for all parallel branches to complete before continuing).

Circle with inner filled circle (bottom) — End point. The workflow is complete.

Activity 1 — Enter repository + policy URL

This is the only user-facing activity. The user types the GitHub repo URL, privacy policy URL, and GitHub token into the form and clicks the button. The workflow does not proceed until all three inputs are provided. FastAPI's Pydantic validation rejects the request immediately if any field is missing — the user sees a validation error and the workflow returns to this activity.

Fork bar — parallel scanning begins

After inputs are validated, the workflow splits into two parallel branches. This is shown by the thick horizontal bar that splits into two arrows going down — one left, one right.

In the actual implementation these two branches do NOT run truly in parallel (that would require Celery or threading). They run sequentially — code scanning first, then policy parsing. However they are shown as parallel in the activity diagram because they are **logically independent** — the output of code scanning does not depend on policy parsing and vice versa. They could run in parallel if async infrastructure were added (which is listed as future work). The diagram shows the **logical** structure, not the physical implementation.

Activity 2a — Scan GitHub repository files (left branch)

PyGitHub authenticates with GitHub using the Bearer token. The full file tree of the repository is retrieved as a flat list. All .py, .js, and .ts files are identified and filtered. Up to 300 files are selected (the current limit — to be removed for final evaluation). For each file, the content is downloaded via GitHub's REST API as base64-encoded text and decoded to UTF-8.

Activity 3a — Regex + Presidio PII detection (two-pass) (left branch continues)

This is the core of the code scanner.

Pass 1 (regex) — every line of every file is checked against 10 regex patterns:

- Aadhaar: aadhaar_number, aadhar_no
- PAN: pan_number, pan_card, pancard
- Phone: phone_number, mobile_no, contact_number
- Email: email_address, user_email
- Location: latitude, longitude, gps, address
- Health: blood_group, diagnosis, disease, medical
- Financial: bank_account, account_no, ifsc, credit_card
- Name: full_name, first_name, last_name, username
- DOB: date_of_birth, birth_date, dob
- Biometric: fingerprint, iris, face_id, biometric

If a line matches, the scanner also checks if any third-party SDK name (firebase, twilio, mixpanel, segment, amplitude etc.) appears on the same line — indicating PII is being sent to an external service.

Pass 2 (Presidio) — every line that was NOT matched by regex is passed to Microsoft Presidio's NER analyzer with spaCy en_core_web_lg. Presidio detects actual PII values (e.g. 9876543210 as a phone number, 2345 6789 0123 as an Aadhaar number, Rahul Sharma as a person name) rather than just field names. Results are mapped to the same PIICategory enum values.

Each matched line from either pass becomes one DataFlowObject with source, file_path, line_number, raw_text, pii_category, sensitivity, and third_party_sdk fields filled in.

Activity 2b — Parse privacy policy text (right branch)

An HTTP GET request is made to the policy URL using the requests library with a standard browser User-Agent header. The raw HTML response is processed:

- <script> blocks are stripped first (to avoid matching JavaScript code as policy text)
- <style> blocks are stripped
- All remaining HTML tags are removed
- Whitespace is normalized

The cleaned text is split into sentences by splitting on ., !, ?, and ;.

Activity 3b — Extract declared PII claims by category (right branch continues)

Each sentence is checked for two things simultaneously:

1. Does it contain a collection context word? (collect, process, store, share, disclose, transfer, use, receive, gather)
2. Does it contain a PII keyword? (name, email, phone, mobile, location, address, health, medical, aadhaar, pan, financial, bank, birth, dob, biometric)

If both conditions are met — the sentence is saying "we collect this type of PII" — it becomes a DataFlowObject with source="policy_parser". Duplicate categories are suppressed — if the policy mentions phone in 3 different sentences, only one DataFlowObject for phone is created.

Join bar — parallel branches merge

The thick horizontal bar appears again to synchronize. The workflow pauses here until both branches are complete — code scanning AND policy parsing must both finish before the next step can begin. In the sequential implementation this join is implicit (the second function call cannot start until the first returns). In a future async implementation, this join would be an explicit await asyncio.gather().

At this point the system has two lists:

- code_findings — all PII found in the source code (e.g. 992 items)
- policy_claims — all PII categories mentioned in the policy (e.g. 6 items)

Activity 4 — Detect consent gaps (code PII − policy claims)

This is the flagship novel contribution of PrivacyPilot — the Consent Gap Detector.

Two Python sets are built:

code_pii = {f.pii_category for f in code_findings}
policy_pii = {f.pii_category for f in policy_claims}

Set subtraction gives the gaps:

gap_categories = code_pii - policy_pii

These are PII types that exist in the code but are NOT mentioned anywhere in the privacy policy — meaning the company is collecting data it never told users about. For the OHC CARE scan this produced {biometric, health, date_of_birth}. For the healthtech-demo scan this produced {aadhaar, pan, biometric, date_of_birth}.

Each DataFlowObject in code_findings then has its is_gap field set to True if its pii_category is in gap_categories.

This entire activity takes microseconds — pure in-memory Python set operations with no I/O.

Activity 5 — Run DPDP Rule Engine (Sec 6, 8(3), 8(4))

The rule engine iterates through every DataFlowObject in code_findings and applies three rules:

Section 6 — Consent Requirements:
If pii_category is in gap_categories (not disclosed in policy):

- Sensitive PII (aadhaar, health, biometric, financial, pan) → CRITICAL violation
- Non-sensitive PII (phone, email, location, name, dob) → HIGH violation

The violation evidence text is auto-generated: "'health' data is collected in the codebase (found at care/emr/admin/patient.py line 32) but is NOT mentioned in the privacy policy. Under Section 6 of the DPDP Act, consent must be obtained before collecting personal data..."

Section 8(3) — Purpose Limitation:
If third_party_sdk is an analytics SDK (firebase, mixpanel, amplitude, segment, crashlytics, clevertap):

- Any PII sent to analytics → HIGH violation (data used beyond stated purpose)
- Sensitive PII sent to any third party → CRITICAL violation (stricter requirements)

Section 8(4) — Data Minimisation:
If sensitive PII is in the code AND not in the policy → HIGH violation (collecting more than necessary)

A seen set prevents duplicate violations — if aadhaar triggers Section 6 once, it won't trigger it again for another occurrence in a different file.

Activity 6 — Calculate compliance score (0–100, deduct per violation)

The scoring formula:

Start at 100

Deduct 15 for each CRITICAL violation

Deduct 8 for each HIGH violation

Deduct 4 for each MEDIUM violation

Deduct 5 for each consent gap

Floor at 0 (cannot go below 0)

The rating is then assigned:

- 80–100 → Good (green)
- 60–79 → Moderate (orange)
- 40–59 → Poor (red)
- 0–39 → Critical (dark red)

For the healthtech-demo scan: 5 critical (×15 = 75) + 7 high (×7 = 56) + 4 gaps (×5 = 20) → would be negative, floored to 0. Score = 0/100 Critical.

Activity 7 — Save scan, findings, violations to PostgreSQL

Everything is written to the database in a single SQLAlchemy transaction:

1. A Scan row is created with summary counts and db.flush() is called to get the auto-generated scan_id without committing yet
2. Up to 200 Finding rows are inserted (capped to keep database lean)
3. All Violation rows are inserted
4. db.commit() is called — this is the single point of persistence

If anything fails during steps 1–4, db.rollback() is called and no partial data is saved. The scan fails cleanly.

Activity 8 — Generate HTML + PDF report with violations + remediation

The report generator builds a full HTML document containing:

- Compliance score gauge with color
- Scan summary (repo, date, counts)
- Consent gaps section with gap tag labels
- DPDP violations section (each with section name, severity badge, evidence, recommended fix)
- Sample code findings table (PII type, sensitivity, file, line, detected by)
- Disclaimer

This HTML is served at *GET /report/{scan_id}*. When *?format=pdf* is added, ReportLab converts the same content into a downloadable PDF using the pdf_generator.py module.

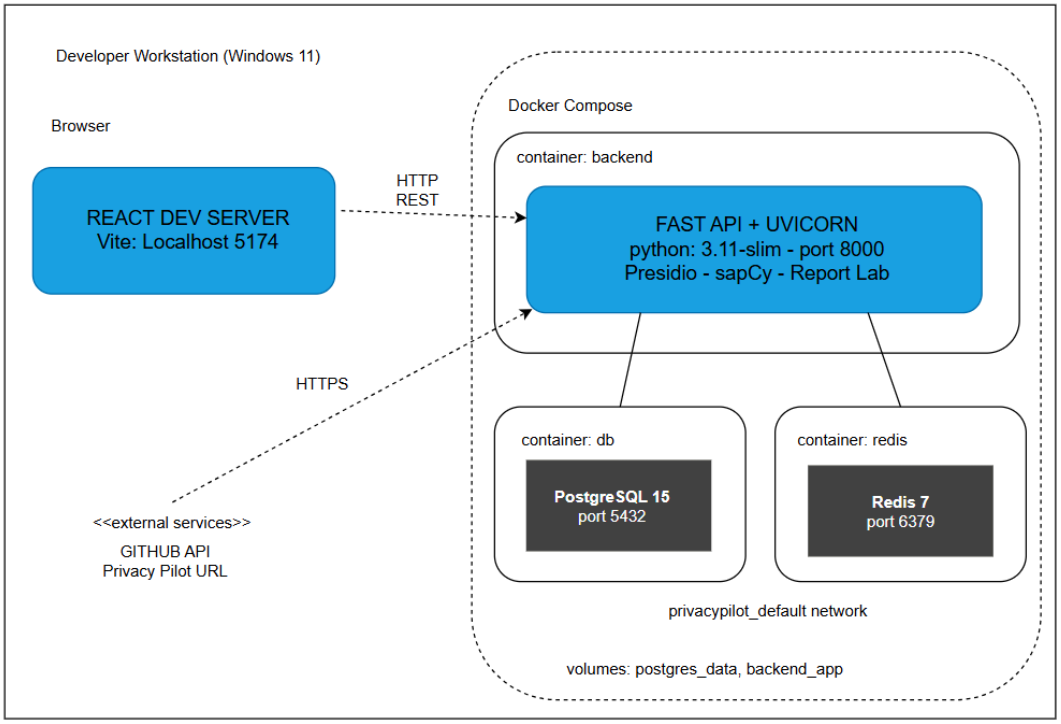
Activity 9 — Display score, violations, download buttons in UI

The React frontend receives the JSON response and renders:

- The compliance score gauge (big colored number, rating label, critical/high count)
- The consent gaps section (yellow tags for each gap category)
- The DPDP violations section (red/orange cards with full evidence and fix text)
- The sample code findings (blue cards with file + line + raw code)
- The "View Full Report" and "Download PDF" buttons

The result.scan_id being present in the response is what triggers the buttons to appear — they link to *localhost:8000/report/{scan_id}* and *localhost:8000/report/{scan_id}?format=pdf.*

7. Deployment Diagram:



A deployment diagram shows **where software actually runs** — the physical and virtual infrastructure, how components are deployed onto machines, and how those machines communicate with each other. Unlike the component diagram (which shows what software exists), the deployment diagram shows **where that software lives at runtime**.

The three key symbols are:

- **Nodes** — physical or virtual machines, shown as 3D boxes or labeled rectangles
- **Artifacts** — actual deployable software units (executables, containers, services)
- **Communication paths** — arrows showing how nodes communicate (HTTP, TCP, internal network)

The entire PrivacyPilot deployment runs on a **single developer workstation** — one Windows 11 laptop or desktop. There is no cloud server, no AWS, no VPS. Everything runs locally using Docker Compose, which creates a private virtual network between 3 containers. The browser accesses the frontend and backend on localhost.

Two external services live outside the workstation — GitHub.com and whatever URL hosts the privacy policy. These are accessed over the internet via HTTPS.

The developer workstation node

This is the outermost box in the diagram. It represents the physical machine — the laptop or desktop where the team develops and runs PrivacyPilot. Key specs from the synopsis:

- **OS:** Windows 11 (development) or Ubuntu 22.04 LTS (deployment)
- **RAM:** 16GB recommended (Docker needs ~3GB: backend ~1.5GB due to spaCy model, PostgreSQL ~300MB, Redis ~50MB)
- **Processor:** Intel Core i7 12th Gen or AMD Ryzen 7 recommended
- **Internet:** 50 Mbps recommended (GitHub API calls during scanning are the bottleneck)

Everything inside this outer box runs on this one machine. There is no networking between two physical machines.

The Browser node

What it is: The user's web browser — Microsoft Edge or Google Chrome — running on the same workstation.

What runs inside it:

React Dev Server — This is the Vite development server started by running npm run dev in the frontend/ directory. It serves the React application at localhost:5174. Key details:

- It is NOT a production build — it uses Vite's hot module replacement (HMR) so code changes in .jsx files reflect instantly in the browser without page refresh
- The React application is a single-page application (SPA) — all routing (home / and /history) is handled client-side by react-router-dom, not by a server
- It runs on port 5174 (not the default 5173) because 5173 was sometimes occupied
- It communicates with the FastAPI backend via fetch() calls to localhost:8000

Important: *The browser node is separate from the Docker Compose node. The browser and React dev server run directly on Windows — NOT inside a Docker container. Only the backend, database, and Redis run inside Docker.*

The Docker Compose node

This is the most important part of the deployment diagram. Docker Compose reads the docker-compose.yml file and creates a private virtual network called privacypilot_default. Inside this network, the 3 containers communicate with each other using their **service names** as hostnames (not localhost). For example, the backend container connects to PostgreSQL using the hostname db, not localhost:5432.

Container 1 — backend

- **Base image:** python:3.11-slim — a minimal Debian-based image with Python 3.11 installed
- **Built from:** backend/Dockerfile
- **Port mapping:** 8000:8000 — the container's internal port 8000 is mapped to the host's port 8000, making it accessible at localhost:8000 from the browser
- **Volume mount:** ./backend:/app — the backend source code directory on the host is mounted into the container at /app. This means when you save a .py file in VS Code, Uvicorn's WatchFiles detects the change and automatically reloads the server — no rebuild needed
- **What runs inside:** FastAPI served by Uvicorn with --reload flag enabled. The full Python environment including all packages from requirements.txt (FastAPI, SQLAlchemy, PyGitHub, Presidio, spaCy, ReportLab etc.)
- **Important:** The spaCy en_core_web_lg model (400MB) must be manually downloaded after every docker compose down using docker compose exec backend python -m spacy download en_core_web_lg — it does not persist unless added to the Dockerfile
- **Environment variable:** DATABASE_URL=postgresql://pilot:pilot@db:5432/privacypilot — notice @db:5432 uses the service name db, not localhost
- **Depends on:** The db container must start and be healthy before the backend starts

Container 2 — db

- **Image:** postgres:15 — official PostgreSQL 15 image from Docker Hub
- **Port mapping:** 5432:5432 — accessible at localhost:5432 for external tools like pgAdmin or DBeaver
- **Environment variables:**
  - POSTGRES_USER=pilot
  - POSTGRES_PASSWORD=pilot
  - POSTGRES_DB=privacypilot
- **What runs inside:** PostgreSQL 15 database server. On first startup, it initializes the privacypilot database automatically using the environment variables
- **3 tables created on startup:** scans, findings, violations — created by create_tables() called in FastAPI's @app.on_event("startup") handler using SQLAlchemy's Base.metadata.create_all()
- **Data persistence:** PostgreSQL data is stored in a Docker volume (postgres_data). This means data persists across docker compose down and docker compose up — but is lost if the volume is explicitly deleted with docker compose down -v
- **Internal hostname:** db — this is how the backend container reaches it. The connection string postgresql://pilot:pilot@db:5432/privacypilot uses db as the host

Container 3 — redis

- **Image:** redis:7-alpine — Alpine Linux based Redis 7 image. Alpine is used because it is tiny (~7MB vs ~110MB for the full Redis image)
- **Port mapping:** 6379:6379 — accessible at localhost:6379
- **What runs inside:** Redis 7 in-memory data store running in standalone mode (not cluster mode)
- **Current usage:** Used as a caching layer. Future use: Celery background job broker for async scanning (listed as future work in the synopsis)
- **No persistence:** Redis data is stored only in memory by default. If the container restarts, all cached data is lost. This is acceptable for caching — the cache simply rebuilds on the next request
- **Internal hostname:** redis — used by the backend if/when Celery is added

The Docker private network

privacypilot_default is the name Docker Compose automatically gives to the private bridge network it creates. Key properties:

- All 3 containers are automatically connected to this network
- Containers can reach each other using service names as DNS hostnames (db, redis, backend)
- The network is completely isolated from other Docker networks on the system
- From outside Docker (the browser, VS Code), containers are reached only through their mapped ports (localhost:8000, localhost:5432, localhost:6379)
- When docker compose down is run, this network is destroyed. When docker compose up is run again, a new identical network is created

The external services

GitHub API (api.github.com)

- Lives entirely outside the workstation on GitHub's servers
- Accessed over the internet via HTTPS (port 443) from inside the backend Docker container
- Authentication: Bearer token provided by the user in the scan form
- Rate limits: 5000 requests per hour for authenticated tokens. At 300 files per scan, each scan uses 300 API calls. So the user can run approximately 16 full scans per hour before hitting the rate limit
- The Docker container reaches GitHub through the host machine's internet connection — Docker containers share the host's network for outbound internet access

Privacy Policy URL (external web server)

- Any web server anywhere on the internet that hosts the company's privacy policy
- Accessed via HTTPS GET request from the backend container using the requests library
- No authentication required — must be a publicly accessible URL
- Known limitation: If the page requires JavaScript rendering (React SPA, Next.js etc.) the requests library will only receive the empty HTML shell, not the rendered content. This is why JavaScript-heavy policy pages return 0 findings

---

The 4 communication paths

Path 1 — Browser → Backend (HTTP REST)
localhost:5174 (React) → localhost:8000 (FastAPI)

This is a standard HTTP/1.1 connection from the browser to the backend. All API calls go through this path. CORS middleware in FastAPI explicitly allows requests from localhost:5173 and localhost:5174. The path goes from the browser process on Windows → through the Windows loopback interface → into the Docker port mapping → into the backend container.

Path 2 — Backend → PostgreSQL (TCP/PostgreSQL protocol)
backend container → db container via privacypilot_default network

SQLAlchemy uses the psycopg2-binary driver to connect to PostgreSQL using the connection string postgresql://pilot:pilot@db:5432/privacypilot. All database operations (INSERT, SELECT, COMMIT, ROLLBACK) go through this internal Docker network path. This path never leaves the workstation — it is purely internal.

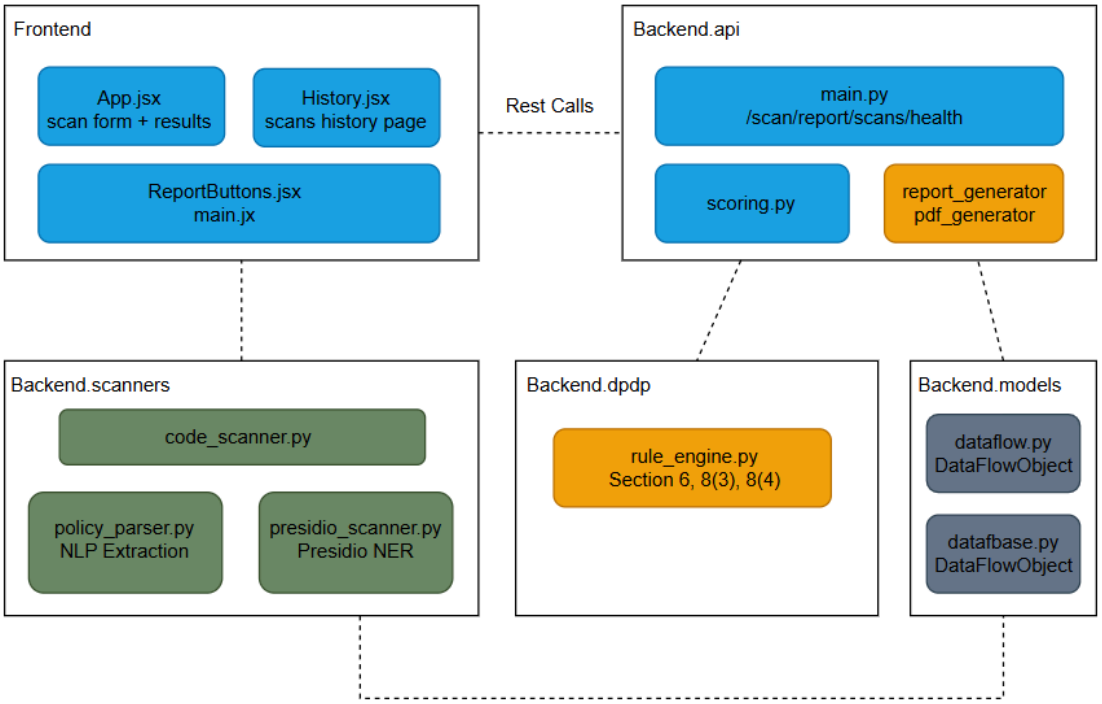
Path 3 — Backend → Redis (Redis protocol)
backend container → redis container via privacypilot_default network

Redis connection using the Redis wire protocol on port 6379. Currently used for caching. This path is also purely internal to Docker.

Path 4 — Backend → External Services (HTTPS)
backend container → internet → api.github.com and privacy policy URL

This is the only path that goes outside the workstation. The Docker container sends outbound HTTPS requests through the host machine's network interface and internet connection. Responses come back the same way. This is the path that determines scan speed — limited by internet bandwidth and GitHub API rate limits.

8. Package Diagram



A package diagram shows **how the codebase is organised into logical groups** — which files belong together, what each group is responsible for, and how groups depend on each other. Think of it as a bird's-eye view of the folder structure, but with meaning attached to each folder — not just "files that happen to be in the same directory" but "files that serve the same architectural purpose."

In UML, a package is shown as a folder tab rectangle. Dependencies between packages are shown as dashed arrows with <<use>> labels — meaning "this package needs something from that package to work."

The 5 packages

Package 1 — frontend

What it contains:

- App.jsx — Main scan form and results display
- History.jsx — Scan history page at /history
- ReportButtons.jsx — View Report + Download PDF buttons component
- main.jsx — React Router setup, route definitions

What it is responsible for:
Everything the user sees and interacts with in the browser. This package owns the entire UI layer — input collection, results rendering, navigation, and report access.

Technology: React 18, Vite, react-router-dom

Key design decision: ReportButtons.jsx was extracted as a separate component specifically because the same two buttons (View Report, Download PDF) appear in

both App.jsx and History.jsx. Extracting it into its own file follows the React DRY principle — one component, used in two places, changed in one place.

Depends on: backend.api — every user action eventually results in an HTTP call to the FastAPI backend. Specifically:

- App.jsx calls POST /scan and GET /report/{id}
- History.jsx calls GET /scans
- ReportButtons.jsx opens GET /report/{id} and GET /report/{id}?format=pdf

Package 2 — backend.api

What it contains:

- main.py — FastAPI application, all endpoint definitions, CORS middleware, startup handler
- scoring.py — Compliance score calculator (0–100)
- report_generator.py — HTML report builder
- pdf_generator.py — ReportLab PDF generator

What it is responsible for:
The API surface of the system — receiving HTTP requests, orchestrating the scan pipeline, and returning responses. This is the central coordinator package. It does not do any scanning or rule evaluation itself — it delegates to other packages and assembles the results.

Why scoring and report generation live here:
Both scoring.py and the report generators are not reusable across multiple parts of the system — they exist purely to serve the API response and the report endpoint. They have no business being in the scanners or dpdp packages. Placing them in backend.api keeps them close to where they are called.

Depends on:

- backend.scanners — calls scan_github_repo() and parse_privacy_policy()
- backend.dpdp — calls run_dpdp_rules()
- backend.models — uses Scan, Finding, Violation database models and PIICategory enum

Package 3 — backend.scanners

What it contains:

- code_scanner.py — GitHub repository scanner (regex + Presidio two-pass)
- policy_parser.py — Privacy policy NLP parser
- presidio_scanner.py — Microsoft Presidio NER wrapper

What it is responsible for:
All data ingestion — fetching raw data from external sources (GitHub repos and privacy policy URLs) and converting it into structured DataFlowObject instances that the rest of the system can work with. This package is the only one that talks to the outside world (GitHub API and policy URLs).

Why these three files belong together:
They all do the same category of work — taking raw external input and producing DataFlowObject lists. They are all "readers" — they read data in, classify it, and hand it up. None of them make decisions about compliance — that is the job of backend.dpdp.

Internal relationship between the three files:
code_scanner.py imports and calls presidio_scanner.py internally — Presidio is the second pass inside the code scanner. policy_parser.py is independent. So within this package, code_scanner.py depends on presidio_scanner.py, but policy_parser.py stands alone.

Depends on:

- backend.models — produces DataFlowObject instances using the PIICategory and Sensitivity enums defined there

Package 4 — backend.dpdp

What it contains:

- rule_engine.py — DPDP Act rule evaluation (Section 6, 8(3), 8(4)), DPDPViolation class definition

What it is responsible for:
The legal intelligence of the system. Takes structured DataFlowObject data and applies DPDP Act rules to produce DPDPViolation objects. This is the only package in the system that contains legal domain knowledge — which DPDP sections apply to which data practices, what the severity levels are, and what the remediation recommendations should be.

Why it is its own package:
The DPDP rule engine is the most likely component to grow over time. Currently it covers 3 provisions. When provisions 7, 9, 10, 12, 13 etc. are added (listed as future work), they all go in this package. Isolating it means legal rules can be updated without touching scanners, API code, or the database schema.

One important note about DPDPViolation:
This class is defined in rule_engine.py — it is an in-memory Python class, NOT a database entity. It exists only during the scan. After run_dpdp_rules() returns, main.py converts each DPDPViolation into a Violation database model and saves it. The two are structurally similar but serve different purposes — DPDPViolation is transient processing output, Violation is persistent storage.

Depends on:

- backend.models — reads DataFlowObject instances (produced by scanners) and uses PIICategory enum to classify violations

Package 5 — backend.models

What it contains:

- dataflow.py — DataFlowObject Pydantic model, PIICategory enum, Sensitivity enum
- database.py — SQLAlchemy database models (Scan, Finding, Violation tables), database connection setup, create_tables() function, get_db() dependency

What it is responsible for:
All data definitions for the system — both in-memory data structures (Pydantic) and persistent database structures (SQLAlchemy). This package defines the "language" that all other packages speak. When the scanners produce a DataFlowObject, when the rule engine reads a PIICategory, when the API saves a Violation — they are all using types defined in this package.

Why these two files belong together:
Both dataflow.py and database.py are pure data definition files — they define shapes and structures, not behaviour. Neither one does any scanning, rule evaluation, or API handling. They are the foundation layer that everything else builds on.

dataflow.py details:
Defines 3 things:

1. PIICategory enum — 11 values (aadhaar, pan, phone, email, location, health, financial, name, dob, biometric, unknown)
2. Sensitivity enum — 3 values (sensitive, personal, unknown)
3. DataFlowObject Pydantic BaseModel — 11 fields, all optional except source, raw_text, field_name, pii_category, sensitivity

database.py details:
Defines 4 things:

1. SQLAlchemy engine + session factory (connects to PostgreSQL using DATABASE_URL environment variable)
2. Scan ORM model — maps to scans table
3. Finding ORM model — maps to findings table
4. Violation ORM model — maps to violations table

Depends on: Nothing — this is the foundation package. No other package in the project is imported here. All arrows flow INTO this package, never out of it.

The 5 dependency arrows

Arrow 1 — frontend → backend.api (REST calls)

The only cross-boundary communication that uses HTTP. Every user action in the browser results in a fetch() call to the FastAPI backend. This is the only dependency that crosses a runtime boundary (browser to server). All other dependencies are Python import statements within the same Docker container.

Arrow 2 — backend.api → backend.scanners

main.py imports scan_github_repo from code_scanner.py and parse_privacy_policy from policy_parser.py. These are called sequentially inside the run_scan() endpoint handler. This is the most traffic-heavy dependency — the scanner functions take 2–4 minutes to return.

Arrow 3 — backend.api → backend.dpdp

main.py imports run_dpdp_rules from rule_engine.py. Called after both scanners complete. Fast — milliseconds.

Arrow 4 — backend.api → backend.models

main.py imports Scan, Finding, Violation from database.py (for database operations) and PIICategory from dataflow.py (for type checking). Also imports create_tables and get_db.

Arrow 5 — backend.scanners → backend.models

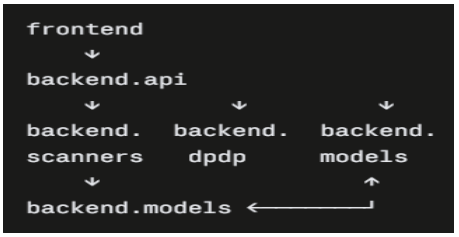
Both code_scanner.py and policy_parser.py import DataFlowObject, PIICategory, and Sensitivity from dataflow.py. Every PII finding they produce is wrapped in a DataFlowObject — so this dependency is used thousands of times per scan.

Arrow 6 — backend.dpdp → backend.models

rule_engine.py imports DataFlowObject and PIICategory from dataflow.py. The rule engine reads the pii_category and third_party_sdk fields of each DataFlowObject to decide which rules fire.

The dependency rule — what it tells you about architecture quality

Notice the direction of all arrows. They all flow in one direction:



backend.models is at the bottom — everything depends on it, it depends on nothing. This is correct architecture — your foundation should have zero dependencies on the layers above it.

backend.scanners and backend.dpdp depend on backend.models but NOT on each other. This means:

- You can upgrade the scanner without touching the rule engine
- You can add new DPDP provisions without touching the scanner
- You can change the database schema in one place (database.py) and it propagates everywhere

backend.api depends on everything below it but nothing above it. The frontend depends on the API but the API has no knowledge of the frontend. This is a clean layered architecture — each layer only knows about the layer directly below it.

If any arrow pointed upward (e.g. backend.models importing from backend.api) that would be a circular dependency — a serious architectural flaw. There are no upward arrows in this diagram.